

Mingtao's Java World

Monday, 15 October 2012

What does ContextLoaderListener do in Spring?

In **web.xml**, there is usually the following line

```
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
```

So what does **ContextLoaderListener** do exactly?

As the Spring API puts it:

Bootstrap listener to start up Spring's root WebApplicationContext. Simply delegates to ContextLoader.

I believe this sentence is too vague to visualize the responsibility this class takes on. So this article tries to break down the duties performed by this class.

In summary, **ContextLoaderListener** does three main tasks:

- **Create a web application context**
- **Read bean definitions from application context xml**
- **Instantiate and initialize beans based on the bean definitions and save the beans into the web application context.**

I will give detailed explanation one by one.

1. Create a web application context

Upon the start of the deployment, callback method **ContextLoaderListener.contextInitialized()** will be invoked by the container or application server.

ContextLoaderListener.contextInitialized() will delegate the task of creating web application context to *this.contextLoader*.

ContextLoaderListener @Line 111

```
this.contextLoader.initWebApplicationContext(event.getServletContext());
```

ContextLoader.java @Line 281

```
this.context = createWebApplicationContext(servletContext);
```

After this line, the web application context is created. Pretty straightforward, isn't it? If we put the break point at the next line, we will see the class of *this.context* is **XmlWebApplicationContext**.

One thing needs probing here:

Who gets to pick this particular implementation of the **WebApplicationContext**?

Spring has provided more than one implementations, including **AnnotationConfigWebApplicationContext**, **GenericWebApplicationContext** and **StaticWebApplicationContext**. How does **XmlWebApplicationContext** distinguish itself among all the candidates? Can we use our own implementation of **WebApplicationContext** should the need arises?

We need to step into the **createWebApplicationContext()** method.

ContextLoader.java @Line 333

```
Class<?> contextClass = determineContextClass(sc);
```

Step into this method, **ContextLoader.java @Line 398**

```
String contextClassName = servletContext.getInitParameter(CONTEXT_CLASS_PARAM);
```

contextClassName is null because there is no such init parameter defined in **web.xml**

Then we have to fall back to the default value, **ContextLoader.java @Line 411**

```
contextClassName = defaultStrategies.getProperty(WebApplicationContext.class.getName());
```

The variable `defaultStrategies` is a **Properties** object, which holds a name-value pair:

```
{org.springframework.web.context.WebApplicationContext=org.springframework.web.context.support.XmlWebApplicationContext}
```

When is the `defaultStrategies` object loaded with this name-value pair and where is this information stored initially?

ContextLoader.java @Line 164-165

```
ClassPathResource resource = new ClassPathResource(DEFAULT_STRATEGIES_PATH, ContextLoader.class);
defaultStrategies = PropertiesLoaderUtils.loadProperties(resource);
```

The value of the constant `DEFAULT_STRATEGIES_PATH` is `ContextLoader.properties`, which resides at the same package of the same jar file (spring-web.jar) as `ContextLoader` does.

We can write our own implementation of **WebApplicationContext** and add a **context-param** entry to **web.xml**.

```
<context-param>
  <param-name>contextClass</param-name>
  <param-value>
    org.springframework.web.context.support.CustomerWebApplicationContext
  </param-value>
</context-param>
```

2. Read bean definitions from application context xml

In this part, 3 questions will be addressed.

1. Where in `ContextLoader` does reading bean definitions take place?
2. We know that the **context-param** entry for **contextConfigLocation** is optional, so where is the default context config location defined?
3. If we do add a **context-param** entry for **contextConfigLocation** in **web.xml**, where does the default value gets overwritten?

ContextLoader.class @Line 284

```
configureAndRefreshWebApplicationContext((ConfigurableWebApplicationContext)this.context, servletContext);
```

Reading bean definitions takes place here. OK, to be fair, lots of things take place here, also including instantiating and initializing beans.

We have to be a bit more specific about the location.

Let's step into this method. **ContextLoader.class @Line 385**

```
wac.refresh();
```

Further step into this method. The actual method body of `refresh()`; is in the super class of `XmlWebApplicationContext`: **AbstractApplicationConext**. Let's jump to **Line 437**.

```
ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();
```

This line does more than merely obtaining a bean factory as the method name suggests. It is also precisely where loading the bean definitions from the application context xml occurs. However, it won't instantiate the beans and put them to web application context yet.

Let's step into this method. **AbstractApplicationContext.java @Line 526**.

```
refreshBeanFactory();
```

Further step into this method. **AbstractRefreshableApplicationContext.java @Line 128**.

```
DefaultListableBeanFactory beanFactory = createBeanFactory();
```

This line creates a bean factory. It's empty yet. If you watch the variable in the debugger, you will find the `beanDefinitionMap` and `beanDefinitionNames` are empty. Step down to **Line 131**.

```
loadBeanDefinitions(beanFactory);
```

This line, unsurprisingly, loads all the bean definitions.

Assume we don't have a **context-param** entry for **contextConfigLocation** in **web.xml**, we will eventually hit **AbstractRefreshableConfigApplication.java @Line 100**

```
return (this.configLocations != null ? this.configLocations : getDefaultConfigLocations());
```

`this.configLocation` = null because it's not defined in **web.xml**. So it falls back to default location, which is defined at **XmlWebApplicationConext @Line 65**.

```
public static final String DEFAULT_CONFIG_LOCATION = "/WEB-INF/applicationContext.xml";
```

This tells us if you don't want to specify the `contextConfigLocation` context parameter in `web.xml`, you need to place the application context file under **WEB-INF**, and name it exactly -- **applicationContext.xml**

Now let's add a `context-param` entry for `contextConfigLocation` in `web.xml`,

```
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>
    /WEB-INF/config/web-application-config.xml
  </param-value>
</context-param>
```

Restart deployment, **ContextLoader.java @Line 380-383**

```
String initParameter = sc.getInitParameter(CONFIG_LOCATION_PARAM);
if (initParameter != null) {
    wac.setConfigLocation(initParameter);
}
```

The above line sets the path of the application context xml file.

Let's jump out of the method and go back to **AbstractApplicationContext.java @Line 440**, and put a break point there.

```
prepareBeanFactory(beanFactory);
```

If you expand object `beanFactory` in the debugger, and inspect the instance variable `beanDefinitionMap` and `beanDefinitionNames`, you will find they are populated with values.

3. Instantiate and initialize beans based on the bean definitions and save the beans into the web application context

Now it is time we threw some bean definitions into the application context xml file and see how Spring instantiates them.

Let's start with something very simple. In the `applicationContext.xml`, add

```
<bean class="org.springframework.web.servlet.handler.BeanNameUrlHandlerMapping"/>
```

Now restart the deployment process, put a break point at **AbstractApplicationContext.java @Line 465**.

```
finishBeanFactoryInitialization(beanFactory);
```

All the beans are created here.

Step into the method, and pause at **AbstractApplicationContext.java @Line 917**

```
beanFactory.preInstantiateSingletons();
```

Step into it, put a break point at **DefaultListableBeanFactory.java @Line 564**

```
RootBeanDefinition bd = getMergedLocalBeanDefinition(beanName);
```

Inspect the `beanName` variable, we are only interested in it when it becomes **'BeanNameUrlHandlerMapping'**

Then it hits **DefaultListableBeanFactory.java @Line 585**

```
getBean(beanName);
```

Given the bean name, the bean is instantiated through reflection. I was once puzzled by the fact that the returned bean is not assigned to any variable for further processing (like storing it to web application context). Would the bean be garbage collected?

If you debug into the `getBean()` method, you will reach **AbstractAutowireCapableBeanFactory @Line 507-510**.

```
addSingletonFactory(beanName, new ObjectFactory() {
    public Object getObject() throws BeansException {
        return getEarlyBeanReference(beanName, mbd, bean);
    }
});
```

Here, the created bean is put into the web application context. Therefore, the bean won't be garbage collected.

2 comments:

 **Maulik Patel** 5 April 2015 at 03:50
Excellent Info.... Thanks man.
[Reply](#)

 **Unknown** 12 May 2016 at 02:22
WOW!
[Reply](#)

Enter your comment...

Comment as:

Unknown (Google)

Sign out

Publish

Preview

☐ Notify me

[Newer Post](#) [Home](#) [Older Post](#)

Subscribe to: [Post Comments \(Atom\)](#)

Followers
Followers (0)

[Follow](#)

About Me
[Mingtao's Java Blog](#)
[View my complete profile](#)

- Blog Archive**
- 2016 (73)
 - 2015 (1)
 - 2014 (3)
 - 2013 (1)
 - ▼ 2012 (20)
 - December (1)
 - November (2)
 - ▼ October (9)
 - Oct 22 (1)
 - Oct 21 (1)
 - Oct 17 (1)
 - Oct 16 (1)
 - ▼ Oct 15 (2)
 - What does ContextLoaderListener do in Spring?
 - [What does webflow:flow-registry tag do?](#)
 - Oct 10 (1)
 - Oct 08 (1)
 - Oct 04 (1)
 - September (7)
 - July (1)
 - 2011 (28)